Task 1:

For the deep learning framework, I chose PyTorch due to its flexibility and ease of use, which makes it ideal for experimentation.

For the transformer backbone, I chose distilbert-base-uncased because it is lightweight and fast. It is a condensed, roughly 40% smaller, and 60% faster version of BERT. Bert-based or Roberta-based approaches are good choices when working with domain-specific text, fine-grained semantic distinctions, or long texts.

When creating the sentence transformer model, I made several design decisions outside of the transformer backbone, which significantly impacted how sentence-level meaning is represented, transformed, and captured. My goal was to develop a lightweight, general-purpose sentence encoder that produces semantically rich, fixed-length embeddings suitable for similarity comparison, or as input features for other tasks such as classification.

```
# Loading the transformer encoder
self.transformer = AutoModel.from_pretrained(transformer_name)
```

This part loads the transformer, it's frozen in design.

The first choice I had to make was to learn how to extract a fixed-size vector from the transformer model's variable-length output. A 3D tensor of shape [batch_size, sequence_length, hidden_size] is produced by DistilBERT, indicating that every token in the sentence has a unique vector. However, we only need one embedding per sentence for most tasks, rather than individual token vectors. Mean pooling, which weights all token embeddings in a sentence, is what I used for this.

```
# Expanding mask for broadcasting
mask_expanded = attention_mask.unsqueeze(-1).expand(hidden_states.size()).float()

# Applying attention mask and compute the sum of valid token embeddings
masked_sum = torch.sum(hidden_states * mask_expanded, dim=1)

# Avoiding division by zero in case of all-masked input
token_counts = torch.clamp(mask_expanded.sum(dim=1), min=1e-9)

# Computing mean pooled sentence embedding
pooled_output = masked_sum / token_counts
```

For several reasons, I went with mean pooling instead of other options. It is straightforward, robust across sentence lengths, and parameter-free. Mean pooling does not presume that any token has greater significance than others, unlike the [CLS] token used for classification in some models. Instead, it provides a comprehensive reflection of the entire sentence. The advantage is that it avoids overfitting to sentence structures or particular token positions. But the disadvantage is that it treats all tokens equally, which might underrepresent more informative words. I also considered attention pooling, which learns token-level importance weights, and max pooling, which captures the most activated features throughout the sequence. However, they require task-specific tuning and introduce learnable parameters. Mean pooling achieved the best balance between ease of use and efficiency for my objective of developing a general-purpose sentence encoder.

```
# Adding Linear layer to project to desired embedding dimension
self.projection = torch.nn.Linear(self.transformer.config.hidden_size,  out_features: 256)
```

After pooling, I added a linear projection layer to map the pooled embedding from the transformer's initial hidden size (768) to a lower-dimensional space (256). This choice was driven by the objectives of the model design as well as practical limitations. First, for tasks like real-time inference or large-scale similarity search, smaller embeddings are more effective. They speed up techniques like cosine similarity and lower memory usage. Second, the projection layer enables the model to modify the raw transformer output by acting as a learned transformation. The projection layer assists in reshaping and reweighting the pooled vector, which optimizes it for downstream tasks even though it contains general-purpose features. In addition, I considered whether to retain the original embedding size or employ an MLP (multi-layer perceptron) head with extra non-linearities. At the end, I determined that a single linear layer offered a decent compromise between simplicity and expressiveness. An MLP could offer further transformation capacity but at the expense of complexity and increased risk of overfitting without sufficient fine-tuning data.

```
# Adding Non-linear activation for output embeddings
self.activation_fn = torch.nn.Tanh()
```

I used a Tanh activation function after the projection layer. I made this decision because I wanted to use these embeddings for task 2 based on cosine similarity. Tanh compresses every vector element into the interval [-1, 1], which is similar to cosine similarity range. This

normalization makes the embedding space more stable, bounded, and comparable across inputs. Although alternatives like ReLU or GELU are frequently employed in transformer internals and deep learning, they lack this bounded property. They can generate extensive, unbounded outputs, which can be problematic for metrics that rely on distance.

This design reflects the fact that the model is a feature extractor that generates embeddings that can subsequently be utilized for tasks like clustering, nearest-neighbor retrieval, or passed into another classifier. I tried to maintain flexibility by focusing the model on producing embeddings.

Task 2:

```
# Loading transformer encoder and tokenizer
self.backbone = AutoModel.from_pretrained(transformer_model_name)
self.tokenizer = AutoTokenizer.from_pretrained(transformer_model_name)
```

I started by maintaining the transformer backbone as a shared encoder between the two tasks to expand the original sentence transformer model for multi-task learning. Reusing the pre-trained transformer for feature extraction made sense because contextualized token embeddings are essential to NER and sentence classification. However, I had to create the output architecture to branch into task-specific heads, one for sentence-level predictions and another for token-level predictions to support both tasks adequately.

```
# NER token classification head
self.ner_output = nn.Linear(self.backbone.config.hidden_size, num_ner_labels)
```

I added a separate linear head for token classification to this architecture so that it would also function for NER. This head is applied to the transformer's raw token-level outputs, such as the last_hidden_state. This head generates a vector of logits, one for each potential entity label, using the hidden state of each token. Because NER requires a label for each token and not just the sentence as a whole, I avoided pooling.

```
# Mapping classification labels to indices
self.cls_label_to_id = {label: idx for idx, label in enumerate(classification_labels)}
self.cls_id_to_label = {idx: label for label, idx in self.cls_label_to_id.items()}

# Mapping NER labels to indices
self.ner_label_to_id = {label: idx for idx, label in enumerate(ner_labels)}
self.ner_id_to_label = {idx: label for label, idx in self.ner_label_to_id.items()}
```

I initialized the model with label mappings. I created two mappings for each task: id2label, which converts predictions back to label names during inference, and label2id, which, during training, converts human-readable labels to numeric IDs. Since all label handling is integrated into the class, no external mapping logic is required at inference time, giving the model flexibility. Additionally, it makes it simple to switch between languages or datasets with various label sets.

```
if task == "classification":...

elif task == "ner":...
```

I added a new argument, task, to the forward() method that controls whether the model should operate in NER or sentence classification modes. If classification is the task, the model first applies mean pooling to the token embeddings before passing the pooled vector through the classification head, Tanh activation, and a dense layer. If NER is the task, the model applies the token classifier head to the complete sequence of token embeddings without pooling. To filter out padding tokens later during prediction or loss computation, I also ensured the NER forward path returned the attention mask and the logits.

```
# Function for classification predictions
def decode_classification_logits(logits, id_to_label):...

# Function for NER predictions
def decode_ner_logits(logits, attention_mask, id_to_label):...
```

Optional: To support this architecture, I created two helper functions to translate the raw logits into comprehensible predictions. I used softmax and argmax to determine the predicted class and confidence score for sentence classification. I then used id2label to map the expected index to a label name. I used argmax to choose the most likely label for each token, applied softmax over token logits for NER, and filtered out padding tokens

using the attention mask. These helpers facilitate the deployment or evaluation of the model by neatly encapsulating the decoding logic.

Example: To obtain sentence-level labels and confidence scores for sentence classification, we can enter a list of sentences and set task=" classification." For NER, we can use the attention mask to exclude padding to transform the token-level logits into a series of labels after passing a sentence and specifying task="ner". To obtain one label per original word, we can further process token-level NER predictions using a different helper function. I didn't implement it, though, because this task aimed to concentrate on the architecture.

Task 3:

Implications of Freezing:

I discovered that the choice to freeze or adjust specific elements had a direct effect on the model's generalization, training stability, and eventually, overall performance.

- **If the entire network should be frozen:**

By freezing the entire network, including the transformer and task-specific heads, the model effectively transforms into a static feature extractor. This configuration may be helpful when inference speed is crucial or there is not enough data to enable meaningful training. However, it restricts the model's flexibility to the particular task domain. Performance relies entirely on the general-purpose features acquired during pretraining because no network component updates its weights and biases. This method is usually applied to downstream models that depend on fixed embeddings or rapid prototyping.

- **If only the transformer backbone should be frozen:**

A popular strategy that is used by many scientists in transfer learning is to freeze the transformer while letting the task-specific heads train. While the heads focus on task-relevant distinctions, the backbone offers contextual representations from extensive pretraining. This configuration maintains the pre-trained model's capacity for generalization while cutting down on training time and GPU memory usage.

- **If only one of the task-specific heads (either for Task A or Task B) should be frozen:**

It may be beneficial to freeze the head of a task if it is already operating efficiently and any additional updates could actually degrade its performance. In this way, the transformer and the other task head can continue to learn without interfering with the already stable one. For example, we might freeze that head and keep training NER if sentence classification has already performed well after being trained on a sizable dataset. However, since the transformer is still shared by both heads, any modifications to it may still affect both tasks, so this strategy must be used carefully.

Applying Transfer Learning:

- **The choice of a pre-trained model:**

I realized that choosing the best pre-trained model really comes down to two factors: the model's degree of domain alignment and the resources that are available. Models such as distilbert-base-uncased and bert-base-uncased perform surprisingly well and provide a strong baseline for general tasks. However, domain-specific models like BioBERT or LegalBERT can significantly improve processing of specialized text, such as biomedical or legal documents. In many cases, faster convergence and more meaningful representations result from starting with a model that is already tuned to the context of the task.

- **The layers you would freeze/unfreeze:**

I've discovered that training only the task-specific heads at first, with the transformer encoder frozen, works well for transfer learning. This enables the heads to learn how to interpret those embeddings for the particular tasks, while the model maintains the general language comprehension it acquired during pretraining. Unfreezing the transformer and fine-tuning it, typically with a lower learning rate, can help the model better adapt to domain-specific language patterns without forgetting what it already knows once the heads are operating efficiently.

Task 4:

I created a training script for a multi-task natural language processing (NLP) model that can recognize named entities (NER) and classify sentences. I aimed to build a clear and

compelling training loop that allows each task-specific head to learn independently while maintaining a shared transformer backbone. I selected this structure because it promotes knowledge sharing and parameter efficiency across tasks. The model accepts two kinds of input: token-level data for NER (which produces a label for each token) and sentence-level data for classification (which makes a label per sentence). The loop switches between the two tasks, and every epoch during training computes their losses independently and then adds them to update the model overall.

Also, I made some presumptions to keep the training logic precise and controllable. For each task to be completed independently, I first assumed there would be two distinct PyTorch DataLoaders: one for NER and one for sentence classification. These DataLoaders were configured to produce batches specific to their tasks: raw sentences with class labels for classification and tokenized inputs with token-level labels for NER. Additionally, I thought that all required preprocessing had already been finished before training started. I ensured the model was configured with internal label2id and id2label mappings to streamline label handling during training and assessment. By making these decisions, I could concentrate on creating a strong training loop without being distracted by runtime label alignment or preprocessing.

I started the training loop by placing the model in training mode and iterating over batches from the NER and classification DataLoaders simultaneously using Python's zip() function. To maintain a predictable and debug-friendly flow, I first processed the sentence classification task in each batch pair. The model receives raw sentences and produces logits when the task flag is set to "classification." A typical cross-entropy loss is then used to compare these logits to the actual class labels.

After processing classification, the loop proceeds to the NER batch, which generates token-level logits. I had to include only valid (non-padded) tokens in the loss calculation because token sequences are padded to produce consistent batch sizes. To deal with this, I flattened the logits and labels after using the attention mask to filter out the padded positions. This way, padding won't skew the learning process because the cross-entropy loss is only calculated on the active tokens.

Also, I combined the losses from the classification and NER tasks using a simple additive loss function. To update the shared transformer and the task-specific heads in a single optimizer step, the plan is to add them up and then backpropagate the total loss. This allows both tasks to impact the model at each iteration while maintaining a simple training loop.